

Programmation Backend

Sessions et Cookies



Pr. ELALAOUI Hasna

h.elalaoui@uca.ac.ma

1

Les cookies

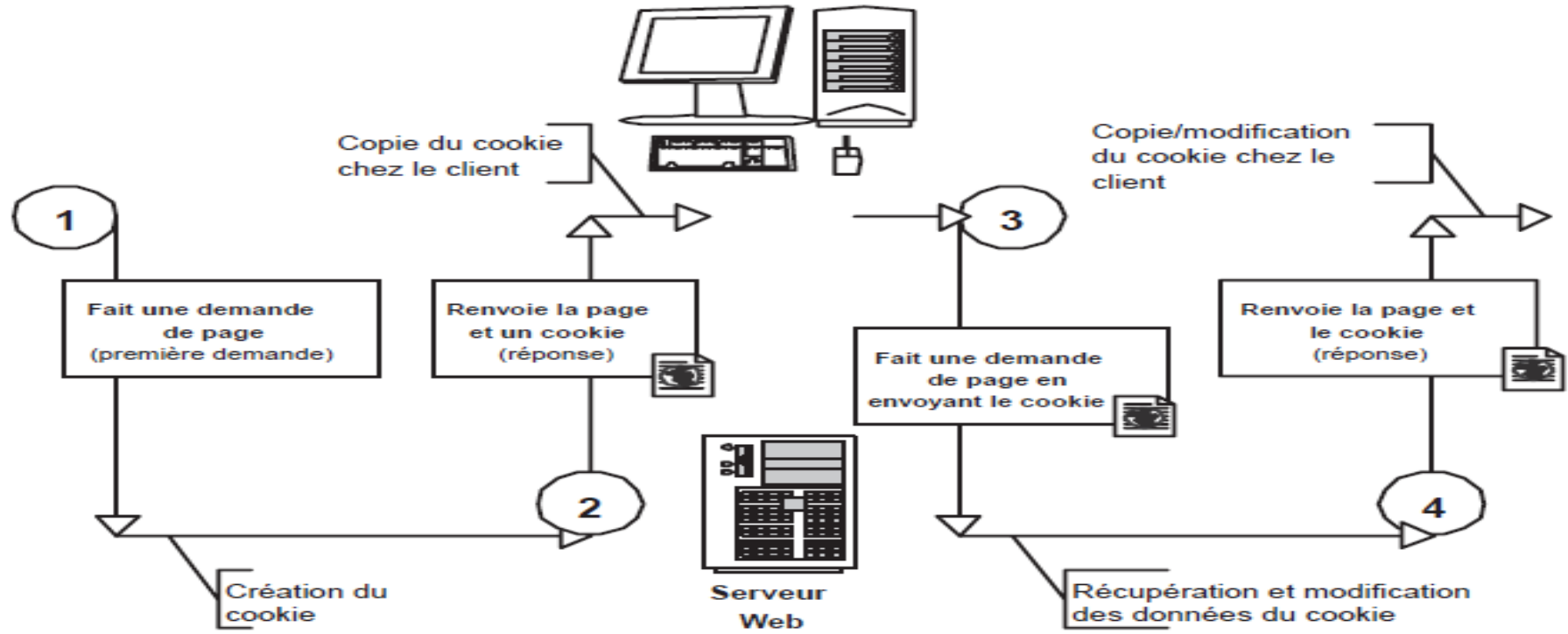


Exemple problématique

- On veut développer une petite application web qui demande à l'utilisateur de s'authentifier seulement si ce dernier se connecte pour la première fois ou d'un nouvel ordinateur, si non il accède directement à la page principale du site .
- La question qui se pose est:
 - Comment on va différencier les utilisateurs ?
 - comment savoir si l'utilisateur à déjà fait une authentification ou pas ?
- Solution → Les cookies



Principe général





Qu'est ce qu'un **cookie**?

- Un cookie est un **fichier texte** stocké chez le client (sur son disque dur).
- Il est généré par le site Web visité et lui permet d'enregistrer des informations en local chez le client pour qu'il les récupère après.
- Les cookies servent généralement à enregistrer les préférences de l'utilisateur ou à stocker des données spéciales comme les identifiants de sessions.
- Les cookies font partie des spécifications du protocole HTTP. En effet, l'entête d'une requête ou d'une réponse HTTP prend en compte la lecture et l'écriture des cookies.



Exemple

- Pour mieux comprendre l'utilité des cookies, supposons qu'un site Web peut être présenté en trois thèmes différents: **bleu**, **vert** ou **rouge**.
- Si le client visite le site pour la première fois alors celui ci est affiché avec le thème bleu (thème par défaut).
- Cependant, une option permet au visiteur de choisir le thème avec lequel il souhaite désormais poursuivre la navigation.
- Si le client choisi le thème vert, alors le design du site bascule aussitôt au thème sélectionné. S'il quitte le site et qu'il revient le visiter une prochaine fois (**en utilisant le même navigateur**) le thème vert est automatiquement chargé.
- En fait, au moment où le client a sélectionné le thème vert, cette option est ajoutée au cookie déposé sur son système par le site.
- Mais si le client décide d'effacer les cookies alors aucune trace du thème choisi ne sera trouvée et le site se chargera la prochaine fois avec le thème par défaut qui est bleu.



Création d'un cookie

○ Syntaxe :

```
<?php  setcookie( nom_de_cookie , valeur_cookie , duree_cookie );  ?>
```

○ Exemple

```
<?php  setcookie ("login" , "user@gmail.com" , time() + 365*24*3600 );  ?>
```

○ L'exemple ci-dessus créera un cookie avec le nom "login" et la valeur " user@gmail.com" avec une durée d'un an.

○ La fonction **setcookie()** doit être avant tout code HTML





Lire un cookie

- Avant de commencer à travailler sur une page, PHP lit les cookies du client pour récupérer toutes les informations qu'ils contiennent.
- Ces informations sont placées dans la **superglobale** `$_COOKIE`, sous forme d'un tableau associatif.
- Syntaxe: afficher la valeur d'un cookie

```
<?php echo $_COOKIE['nom_cookie']; ?>
```

- Afficher la valeur de cookie de l'exemple précédent :

```
<?php echo $_COOKIE['login']; ?>
```




Écriture/Lecture **tableau**

- Pour stocker des types autre que les nombres ou chaînes de caractères, il faudra transformer ces données en une chaîne de caractères avant d'envoyer le cookie : sérialisation
- Deux fonctions PHP :
 - `serialize()` sert à transformer une variable en une chaîne de caractères avant d'envoyer le cookie.
 - `unserialize()` fait l'opération contraire, une fois le cookie récupéré

```
<?php
//Écriture d'un cookie avec un tableau comme valeur
$stab['nom'] = "kassimi";
$stab['prenom'] = "ahmed";
$strTab = serialize($stab);
setcookie("cook",$strTab,time()+3600);

?>
```

```
<?php
//Lecture d'un cookie avec un tableau comme valeur
if (isset($_COOKIE['cook'])) {
    $strTab = unserialize($_COOKIE['cook']);
    echo $strTab['nom'];
}

?>
```



Détruire un cookie



Pour supprimer un cookie, on utilise la fonction **setcookie()** avec en paramètre le nom du cookie à détruire

```
<?php  
//on détruit le cookie nommé moncookie  
setcookie('moncookie');  
?>
```



Une autre méthode consiste à définir une date d'expiration dépassée.

```
<?php  
//on détruit le cookie nommé moncookie  
setcookie('moncookie', '', 0);  
?>
```



Vérifier cookie

- Pour vérifier l'existence d'un cookie sur le poste d'un utilisateur, on utilise la fonction `isset()`
- Exemple

```
<?php
if( isset($_COOKIE['moncookie'])) {
    echo 'Le cookie existe.';
} else {
    echo 'Le cookie n\'existe pas!'; }
?>
```



Exemple récapitulatif

Création d'un cookie

```
<?php
setcookie('langage', 'PHP version 5') ;
?>
<html>
<head><title>titre</title></head>
<body>
<p>Un cookie a été envoyé</p>
<p>Son nom est : langage</p>
<p>Son contenu est : PHP version 5</p>
</body>
</html>
```

Lecture d'un cookie

```
<?php
// On vérifie si le cookie a été reçu
if ( isset($_COOKIE['langage']) ) {
    echo 'Un cookie a été envoyé<br>' ;
    echo 'Son nom est : langage<br>' ;
    echo 'Son contenu est : ' ;
    // On lit la valeur du cookie et on l'affiche
    echo $_COOKIE['langage'] ;
    echo '<br>' ;
} else {
    // Le cookie n'a pas été reçu
    echo 'Aucun cookie du nom de langage n\'a été
reçu<br>' ;
}
?>
```



Limitations des cookies

- Leur taille est limitée
- Le visiteur peut les modifier à loisir.
- Ce n'est donc pas un bon emplacement pour des données sensibles comme des données d'authentification.

2

Les sessions



Exemple problématique

- Imaginez un script qui demande un login et un mot de passe pour qu'un visiteur puisse se « connecter » (s'authentifier), et on veut se souvenir de l'identifiant du visiteur sur toutes les pages du site !
- Restreindre certaines pages de notre site à certains visiteurs uniquement.
- Site de vente en ligne et gestion des paniers (un client peut se déplacer de page en page tout en gardant son panier rempli)
- Solution → Les sessions

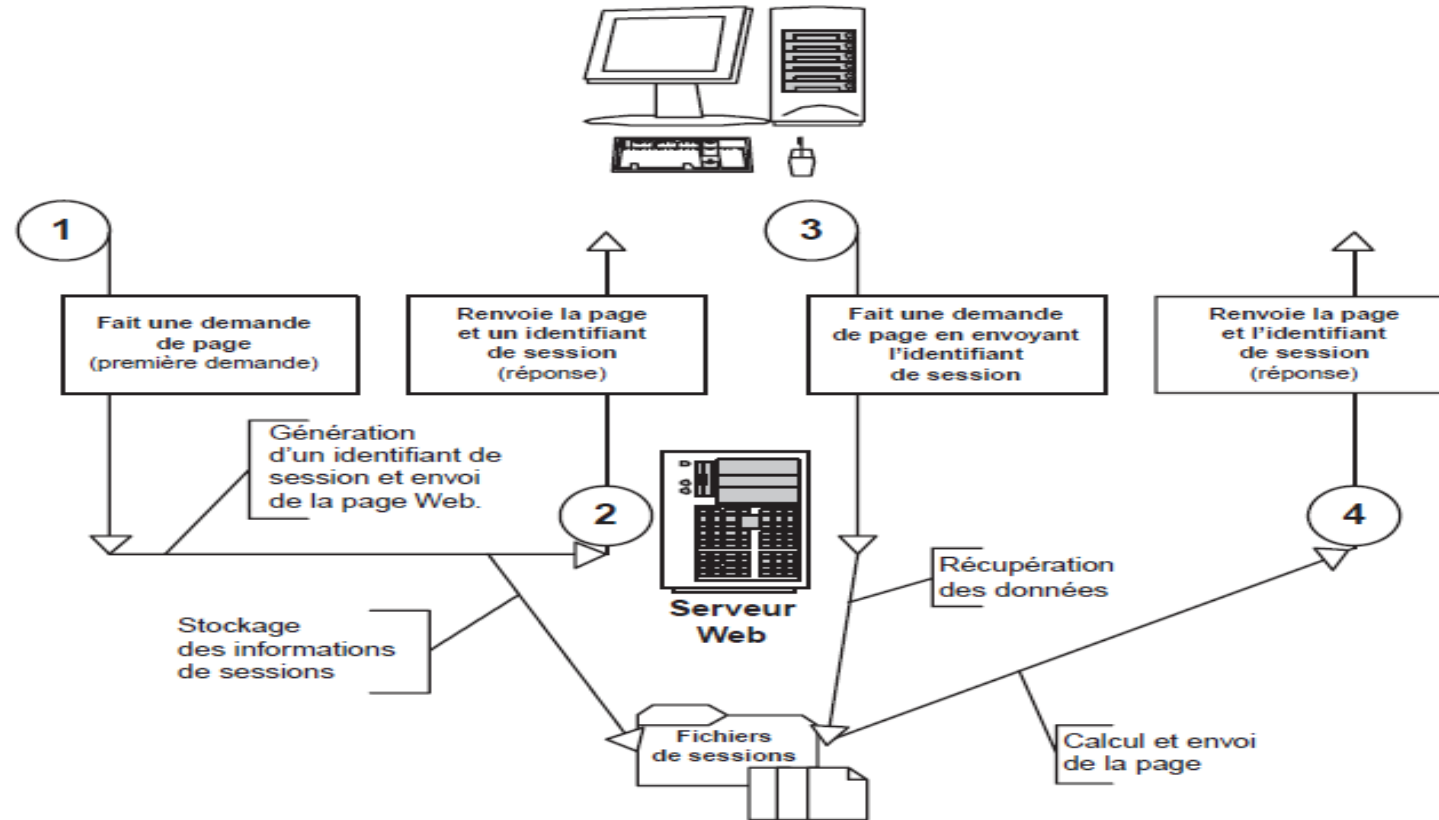


Qu'est ce qu'une session?

- Pour répondre aux limitations des cookies, PHP met à disposition un concept plus évolué : les sessions
- Au lieu de stocker vos informations chez le visiteur, vous les stockez sur le **serveur**.
- Techniquement, vous attribuez au visiteur un **identifiant de session**.
- À chaque fois qu'il revient en annonçant cet identifiant, PHP récupérera toutes les informations relatives à ce visiteur qu'il avait sauvegardées.
- Elles sont très utiles car ils permettent de faire persister des informations aussi longtemps que le client est connecté voir même après qu'il ait quitté le site Web.
- Ces informations sont accessibles par toutes les pages visitées par le client (d'où le nom de session).



Principe général





Session vs. cookie

Cookies	Sessions
Données stockées sur le navigateur du client	Données stockées sur le serveur
Données peuvent être altérées par le visiteur	Données stockées en sécurité
Quantité d'information limitée.	Quantité d'information non limitée.
Avec date d'expiration	Les sessions ne sont pas faites pour durer



Lecture et écriture **session**

- La session s'initialise avec **session_start()** : pour la première requête, PHP crée un identifiant de session **unique** aléatoirement, l'envoie au visiteur et crée le fichier de données correspondant.
- Pour les requêtes suivantes; PHP essaie de lire l'identifiant fourni par l'utilisateur, cherche le fichier correspondant, et vous met à disposition les informations sauvegardées dans la superglobale **\$_SESSION[]**.
- Il s'agit d'un tableau associatif qui contient les valeurs stockées dans les sessions et qui est accessible quelque soit le contexte. Elle est reconnue sur tout le site Web et sa valeur peut être récupérée de n'importe quelle page.
- La lecture, modification ou suppression se fait à travers le tableau **\$_SESSION[]**



Exemple

```
<?php
session_start() ;           // initialisation
// Tester la présence de la variable 'langage' dans la session
if ( isset( $_SESSION['langage'] ) ) {
    echo 'langage existe dans la session et sa valeur est ' ;
// Lecture de la variable de session 'langage'
    echo $_SESSION['langage'] ;
}
else {
    echo 'langage n\'existe pas dans la session' ;
}
?>
```



Remarques importantes

- Contrairement aux cookies, vous n'avez aucune restriction sur le type de données stockées ou la quantité d'information
- Vous pouvez stocker des tableaux, des nombres, du texte ou des objets sans passer par l'étape de sérialisation.
- Exemple

```
<?php  
$tableau = array('un', 'deux', 'trois', 'quatre');  
$_SESSION['tab'] = $tableau;  
?>
```



- L'appel à `session_start()` comme `setcookie()` : doit être placé en haut du script, avant toute sortie vers la page web.
- L'initialisation doit se faire dans **tous les scripts utilisant les sessions**, pas uniquement le premier.



Détruire une session

- La fonction `session_destroy` détruit toutes les données de la session déjà établie.
- Cette fonction détruit la session sur le serveur mais garde toujours le cookie qui lui est associé sur le client.
- Pour supprimer aussi le cookie (cas échéant), on fait appel à la fonction `setcookie()`

```
<?php  
                session_destroy();  
?>
```

Notez que parfois, il est plus pratique de supprimer une donnée particulière d'une session plutôt que de détruire la session elle-même et supprimer par conséquent toutes ses données.